UG CALC · PYTHON LEARNING GUIDE

Object-Oriented Programming

Learn the core OOP ideas by turning your grade calculator into a Python class.

1. What is OOP?

Object-oriented programming (OOP) organizes a program around **objects**. An object keeps related data and the actions that use that data together.

In UG Calc, one calculator object can keep the four grade points and calculate a CGPA and class of degree from them.

> **Think of a class as a recipe and an object as a calculator made from that recipe.** The class describes what it stores and what it can do.

2. The main OOP terms

Class: The blueprint. `CGPACalculator` describes a calculator.

Object: One usable instance of the class. `calculator` is an object containing a student's grades.

Attributes: Data stored on an object, such as `self.level100` and `self.level400`.

Methods: Functions that belong to a class, such as `calculate_cgpa()`.

Constructor: `__init__` runs when an object is created and sets up its starting data.

`self`: Refers to the specific object using a method. It lets that method access the grades stored by that object.

3. How this differs from the original

Your original `syntax.py` asks for input, calculates the result, and prints it in sequence. That is a procedural approach.

With OOP, the grade data and calculation behavior are grouped in `CGPACalculator`. You can create another calculator object with a different student's grades without rewriting the calculation.

> **Formula:** (Level 100 + Level 200 + 2 × Level 300 + 2 × Level 400) ÷ 6

Build the OOP Calculator

This complete program defines the class, asks for the four grade points, creates an object, and prints the result.

```
class CGPACalculator:
    def __init__(self, level100, level200, level300, level400):
        self.level100 = level100
        self.level200 = level200
        self.level300 = level300
        self.level400 = level400

    def calculate_cgpa(self):
        return (
            self.level100
            + self.level200
            + self.level300 * 2
            + self.level400 * 2
        ) / 6

    def get_classification(self):
        cgpa = self.calculate_cgpa()

        if cgpa >= 3.6:
            return "First Class"
        elif cgpa >= 3.0:
            return "Second Class Upper"
        elif cgpa >= 2.0:
            return "Second Class Lower"
        elif cgpa >= 1.5:
            return "Third Class"
        else:
            return "Pass"


level100 = float(input("Enter Level 100 grade: "))
level200 = float(input("Enter Level 200 grade: "))
level300 = float(input("Enter Level 300 grade: "))
level400 = float(input("Enter Level 400 grade: "))

calculator = CGPACalculator(level100, level200, level300, level400)

cgpa = calculator.calculate_cgpa()
classification = calculator.get_classification()

print(f"Your CGPA is: {cgpa:.2f}")
print(f"You are in: {classification}")
```

4. Follow the data through the program

1. `input()` gets each grade as text; `float()` converts it to a number.
2. `CGPACalculator(...)` creates the object. Python calls `__init__` and stores each value on `self`.
3. `calculator.calculate_cgpa()` uses that object's grades and returns a number.
4. `calculator.get_classification()` compares the CGPA with the class boundaries and returns a label.
5. The two `print()` calls show the returned values. `:.2f` displays the CGPA to two decimal places.

5. Try it yourself

Save the code as a separate practice file, for example `oop_calculator.py`. Run it with Python and try different grade points. For a test object without prompts, use:

```
calculator = CGPACalculator(3.8, 3.2, 3.6, 4.0)
print(calculator.calculate_cgpa())
print(calculator.get_classification())
```

This gives a CGPA of **3.70** and the classification **First Class**.

> **Remember:** HTML and JavaScript power the web pages; this lesson practices OOP in the separate Python calculator.

Based on the formula and class thresholds in the UG Calc project's `syntax.py`.